

Lesson 2 — Moving Pictures

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

The screen is upside down

(0, 0) is the **top-left**. **x** grows right. **y** grows **DOWN**.

To move	Write
right	<code>x = x + s</code>
left	<code>x = x - s</code>
up	<code>y = y - s</code>
down	<code>y = y + s</code>

Velocity

Speed **with a direction**. The sign is the direction.

```
speedX = 220    // rightwards
speedX = -220   // leftwards
speedY = -160   // UPWARDS
```

Bounce = **flip the sign**: `speedX = -speedX`

Delta time

```
let lastTime = 0;

function frame(currentTime) {
  if (!lastTime) lastTime = currentTime;
  let dt = (currentTime - lastTime) / 1000;
  lastTime = currentTime;
  if (dt > 0.1) dt = 1 / 60;    // CLAMP

  update(dt);
  render();
  requestAnimationFrame(frame);
}
requestAnimationFrame(frame);
```

Then multiply **every** movement by `dt`:

```
ball.x += ball.speedX * dt;
```

Now the speed is **pixels per second**, not per frame.

distance = speed × time

At 60 fps, `dt ≈ 0.0167`, so `120 × 0.0167 ≈ 2` pixels.

Bouncing, correctly

```
if (ball.x + ball.radius > canvas.width) {
  ball.x = canvas.width - ball.radius; //
FIRST
  ball.speedX = -ball.speedX;          //
THEN
}
```

Test the **edge**, not the centre. Fix the **position** before the **velocity**, or the ball vibrates against the wall.

Words

dt — seconds the last frame took. **Clamp** — force a number into sensible limits. **Radius** — centre to edge.

Section A — Recall

A1. On a canvas, which direction does `y` get bigger in? Write the line of code that moves something *up*. [2 marks]

A2. What does `dt` stand for, and what does it actually measure? [2 marks]

A3. What is the difference between *speed* and *velocity*? [2 marks]

A4. A ball moves at 300 pixels per second. The last frame took 0.02 seconds. How far did it move this frame? Show your working. [3 marks]

A5. Why must you *clamp* `dt`? Give two different situations where an unclamped `dt` would be huge. [3 marks]

Section B — Predict the output

Write your prediction down before running anything.

B1. A game moves a square with `x = x + 4` each frame. On your 60 fps laptop it crosses a 600-pixel canvas in 2.5 seconds. Your friend has a 144 fps monitor. How long does it take on their machine? Show your working. [3 marks]

B2. The ball starts at `y = 100` with `speedY = -90`. Where is it one second later, and is that higher or lower on the screen? [3 marks]

B3. What is drawn here, and where does the ball end up? [4 marks]

```
let dt = (currentTime - lastTime) / 1000;
// lastTime is never updated
update(dt);
```

B4. A student starts their loop with `frame()` instead of `requestAnimationFrame(frame)`. The ball [4 marks]
disappears completely and never comes back. Explain the chain of events. What is the value of `ball.x` after
the first frame?

Section C — Find and fix the bug

C1. The ball reaches the right wall and then shivers against it forever instead of bouncing away. Explain [4 marks]
what happens on each of the first three frames after it touches the wall, then write the fix.

```
ball.x = ball.x + ball.speedX * dt;

if (ball.x + ball.radius > canvas.width) {
    ball.speedX = -ball.speedX;
}
```

C2. The ball sinks halfway into every wall before bouncing back out. The bounce itself works. What is [3 marks]
wrong?

```
if (ball.x > canvas.width) {
    ball.x = canvas.width;
    ball.speedX = -ball.speedX;
}
```

C3. The ball is somewhere near the middle when the page loads, then instantly vanishes. The console [4 marks]
shows no errors at all. What is the most likely cause, and why does it produce no error message?

Section D — Build it

Start from your lesson 1 file, or from `code/02-delta-time.html`.

- ☐ **Checkpoint 1.** Add the `dt` calculation to your loop, including the clamp. Draw `dt` and `Math.round(1 / dt)` on screen so you can see them. What frame rate is your machine running at?
- ☐ **Checkpoint 2.** Replace your square with a circle using `ctx.arc`, and store its numbers in a single `ball` object rather than loose variables.

- ☐ **Checkpoint 3.** Give the ball `speedX` and `speedY` in *pixels per second*, and move it with `* dt`. It should drift diagonally off the screen.
- ☐ **Checkpoint 4.** Make it bounce off all four walls. Remember: test the **edge**, and fix the **position before the velocity**.
- ☐ **Checkpoint 5.** Prove your delta time works. Open the page in two windows, drag one around constantly to make its frame rate wobble, and check both balls stay in step. (They will drift apart eventually — think about why that is *not* the same bug.)
- ☐ **Checkpoint 6.** Add gravity: `ball.speedY = ball.speedY + 600 * dt;` at the top of update. Then make the floor bounce lose energy with `-0.8` instead of `-1`.

IF THE BALL FLIES OFF THE SCREEN THE INSTANT YOU LOAD THE PAGE

You have not clamped `dt`. The first frame is measuring the whole page-load time, so

the ball is being told to move several seconds' worth in one go. This is the single most common bug in this lesson — everybody hits it.

Section E — Design challenge

NO RIGHT ANSWER

Marked on your reasoning. Name what your idea costs, not only what it gains.

Delta time fixes *movement*. But other things in a game can be measured in frames by accident.

E1. For each of these, describe exactly what a player on a 144 fps monitor would experience, then write the fixed version:

1. An enemy that fires every 30 frames.
2. An explosion animation that advances one picture per frame.
3. A countdown that does `secondsLeft = secondsLeft - 1` each frame.

E2. Is there any situation where counting frames genuinely is the right choice? Competitive fighting games describe moves as "7 frames of startup", and tournament players really do count frames. What is different about that situation?

E3. Delta time makes the game fair across machines, but it also makes it **non-deterministic**: run the same game twice with the same button presses and you get slightly different results, because the frame times are never identical. Why might that be a serious problem for online multiplayer, or for a replay feature?

Stretch goals

1. Many balls — if you start writing `ball1`, `ball2`, `ball3`, stop. That feeling is what an **array** is for.
2. A fading trail from the last 20 positions.
3. Draw a live frames-per-second readout. What makes the number move?
4. Make the ball change colour based on how fast it is going.